

軟體供應鏈簽章與可驗證 Provenance

這份文件是給組員製作簡報用的完整交接資料。內容包含題目背景、實作專案、GitHub attestation 證據、每個資料夾用途、Demo 重跑方法、截圖使用建議，以及逐頁簡報規劃。

公開 GitHub Repo

github.com/blizzard-new/software_supply_chain_provenance_demo

最新 CI 狀態

Run [26842529539](#) 已完成，conclusion 為 success。

主要證據 Run

Run [26841214975](#) 是本文件截圖和 terminal 證據的主要來源。

交接資料夾

`team_handoff/` 和 `presentation_materials/` 是組員做簡報時最常用的兩個資料夾。

一句話主軸：我們不是只證明程式碼存在，而是證明最後交出去的 artifact 是由指定 GitHub repo、指定 commit、指定 GitHub Actions workflow 建出來，並且任何一個 byte 被改掉都會驗證失敗。

建議用途：組員先讀第 1 至 6 節建立共識，再直接按照第 9 節逐頁做簡報。

1. 題目定位與報告重點

這個題目屬於現代軟體供應鏈安全。傳統報告常停在「程式碼有沒有簽章」或「檔案 hash 是否相同」，但現在攻擊者更常針對 CI/CD、build script、release artifact、套件上傳流程下手。因此簡報要把重點放在 **artifact** 從哪裡來、誰建的、用哪個 **workflow** 建的、是否被竄改。

報告核心問題：如果使用者下載到一個 wheel、binary 或 container image，他要怎麼知道這個檔案真的來自我們宣稱的 source repository 和 build workflow，而不是攻擊者另外做出來的同名檔案？

1.1 這次實作回答什麼問題

- 如何在 GitHub Actions 裡 build Python package artifact。
- 如何用 actions/attest@v4 為 build output 產生 artifact attestation。
- 如何用 gh attestation verify 驗證 artifact digest、repo、workflow、commit 等 provenance 資訊。
- 如何用 tamper demo 證明「只改 1 byte」就會讓驗證失敗。
- 如何把這些流程整理成可以在期末簡報中展示的證據。

1.2 簡報要避免的誤解

常見誤解	正確說法	簡報怎麼講
有 hash 就等於安全	hash 只能證明內容是否改變，不能證明它是誰建的。	先用 hash 當鋪陳，再說 provenance 補上來源與 build context。
attestation 就是一般簽章	attestation 是帶 metadata 的可驗證聲明，通常會綁定 artifact digest 和 build provenance。	用「簽的是 artifact 加上來源資訊」來解釋。
CI 有過就代表 artifact 可信	CI pass 只代表流程跑完，還要能把下載到的 artifact 綁回那次 build。	展示 verify pass 和 tamper fail 兩張 terminal 圖。
本地 manifest 就是正式 attestation	本地 manifest 是 rehearsal，不是 signed attestation。正式證據來自 GitHub Actions 和 GitHub CLI 驗證。	把 local demo 說成練習版，把 GitHub demo 說成正式版。

2. 技術背景：Artifact、Provenance、Attestation

2.1 名詞解釋

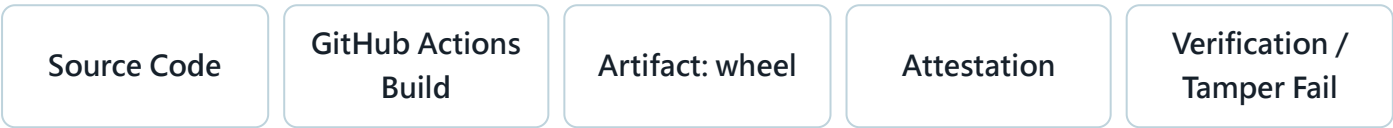
名詞	意思	本專案例子
Artifact	build 流程產出的檔案，可能是 wheel、binary、container image、installer。	dist/hello_provenance_demo-0.1.0-py3-none-any.whl
Digest	artifact 內容的雜湊值。內容變一個 byte，SHA-256 就會改變。	terminal 截圖中的 original sha256 和 tampered sha256。
Provenance	描述 artifact 生產過程的 metadata，例如 repo、commit、workflow、builder。	GitHub Actions 產生的 SLSA build provenance。
Attestation	對 artifact 與 provenance 的可驗證聲明。它讓驗證者確認 artifact 是否符合宣稱來源。	actions/attest@v4 產生的 GitHub artifact attestation。
Verification policy	驗證者接受 artifact 的條件，例如必須來自指定 repo、指定 workflow、digest 要吻合。	scripts/verify_artifact.py 呼叫 gh attestation verify。

2.2 為什麼這是資安題目

軟體供應鏈攻擊不一定要破解加密演算法。攻擊者只要能在 release 流程中替換 artifact、偷改 dependency、污染 build runner、或把惡意檔案偽裝成官方檔案，就可能成功。Provenance 的價值是讓使用者和平台可以問出更精準的問題：

- 這個 artifact 的 digest 是否等於 attestation 裡記錄的 subject digest？
- 這個 artifact 是否由我信任的 repository 產生？
- 這個 artifact 是否由預期的 GitHub Actions workflow 產生？
- 這個 artifact 是否對應到正確 commit，而不是攻擊者自建的分支或 fork？
- 如果 artifact 被改掉，驗證是否會立刻失敗？

2.3 本專案的完整流程



簡報時可以把這個流程講成「從 source 到 artifact 的可驗證鏈」。真正要保護的不是單一檔案，而是 source code、CI/CD、build output、驗證政策 之間的連結。

3. 官方資料整理，可放進簡報的背景

來源	可引用重點	簡報用途
GitHub Docs - Artifact attestations	GitHub Actions 可以為 build artifact 產生建立 build provenance 的 artifact attestation。產生時 workflow 需要設定 id-token: write、contents: read、attestations: write 等權限。	Slide 4 或 Slide 6 說明為什麼 workflow 要這些 permissions。
GitHub actions/attest	actions/attest@v4 支援 provenance、SBOM、custom predicate。沒有提供 SBOM 或 custom predicate 時，預設會產生 SLSA build provenance attestation。	Slide 7 解釋這次 demo 用的是 build provenance 模式。
SLSA v1.2 Provenance	Build provenance 的目標是把 build output 回溯到產生它的 source code 和 build process。	Slide 3 定義 provenance，Slide 10 講限制與延伸。
PEP 740	Python packaging 生態也在推動 index-hosted digital attestations，重點是讓套件 index 能存放和驗證 provenance 相關資訊，而不是只靠舊式 PGP signature。	Slide 4 說明這不是單一平台的玩具，而是套件生態的趨勢。
Sigstore Docs	Sigstore 強調 identity-based signing、OIDC、短期憑證、Fulcio、Rekor transparency log。GitHub artifact attestation 背後也使用 Sigstore bundle 格式。	Slide 4 補充為什麼現代簽章偏向 keyless / identity-based。

簡報語氣建議：不要說「我們發明了一個新的簽章系統」。應該說「我們用 GitHub Actions、GitHub artifact attestations、SLSA provenance、GitHub CLI 驗證，實作一個可展示的軟體供應鏈 provenance demo」。

4. GitHub 與實作狀態

4.1 目前已完成

項目	狀態	證據
公開 GitHub repo	完成	repo home · 截圖 01_repo_home.png
GitHub Actions build	完成	Run 26841214975 和 latest run 26842529539 都是 success
Artifact attestation	完成	03_attestation_created.png · workflow 使用 <code>actions/attest@v4</code>
正式 GitHub verify	完成	11_verify_pass_terminal.png 顯示原始 wheel 驗證通過
Tamper 驗證	完成	12_tamper_fail_terminal.png 顯示竄改檔案驗證失敗
給組員截圖資料夾	完成	team_handoff/screenshots/ 共有 12 張截圖

4.2 重要連結和版本

Repo	https://github.com/blizzard-new/software_supply_chain_provenance_demo
主要證據 run	26841214975 · commit c7ee14ad9699ec7a942ccc481d6be5aa87a622d9
最新 push 後 run	26842529539 · commit 246db4410bf8d55fe586cec56e13a849bff39356
Artifact	<code>python-distributions</code> · wheel 檔名 <code>hello_provenance_demo-0.1.0-py3-none-any.whl</code>

Platform Solutions Resources Open Source Enterprise Pricing

Search or jump to...

Sign in Sign up

blizzard-new / software_supply_chain_provenance_demo Public

Notifications Fork 0 Star 0

<> Code Issues Pull requests Actions Projects Security and quality Insights

← build-and-attest

Organize teammate handoff materials #5

Sign in to view logs

Summary

All jobs

build

Run details Usage Workflow file

Triggered via push 18 minutes ago

blizzard-new pushed c7ee14a main

Status Success

Total duration 33s

Artifacts 1

build-and-attest.yml

on: push

build 27s

Annotations

1 warning

build

Node.js 20 actions are deprecated. The following actions are running on Node.js 20 and may not work as expected: actions/checkout@v4, actions/setup-python@v5, actions/upload...

Show more

Artifacts

Produced during runtime

Name	Size	Digest
python-distributions	8.31 KB	sha256:89325abc3ee0cd5bc171a265e64443cc715effc67d949fc...

可放在簡報第 5 頁：GitHub Actions 已完成 test、build、attest、verify、tamper rejected。

5. 每個資料夾與檔案用途

5.1 資料夾地圖

資料夾	裡面有什麼	組員怎麼用
<code>.github/workflows/</code>	<code>build-and-attest.yml</code> 。定義 GitHub Actions workflow：checkout、setup Python、test、build、upload artifact、generate attestation、verify、tamper rejected。	簡報第 6 至 7 頁可以放 workflow code 截圖，說明正式 GitHub attestation 是在 CI 裡產生。
<code>src/hello_provenance/</code>	Python package source code。這是被 build 成 wheel 的示範 CLI。	簡報只要說明這是 demo artifact 的來源，不需要逐行解釋。
<code>scripts/</code>	<code>create_local_manifest.py</code> 、 <code>verify_artifact.py</code> 、 <code>tamper_demo.py</code> 、 <code>_common.py</code> 。	負責建立 rehearsal manifest、驗證 artifact、製造 tampered artifact。簡報第 8 至 9 頁重點展示 verify 和 tamper。
<code>tests/</code>	CLI unit test。	說明 artifact 不是隨便包出來的，workflow 先跑測試再 build。
<code>presentation_materials/</code>	簡報摘要、逐頁大綱、研究資料、demo script、Q&A、Mermaid 圖、CSV。	組員做 PPT 前可以先讀這裡；如果時間不夠，直接讀本 PDF 第 9 節。
<code>team_handoff/</code>	給組員的交接資料：GitHub evidence、截圖 checklist、訊息草稿、本 PDF、HTML 原稿。	整個資料夾可以直接傳給組員。
<code>team_handoff/screenshots/</code>	12 張簡報可用截圖，含 GitHub 頁面、Actions、code、terminal。	簡報圖片來源。每張圖的用途在第 7 節列出。
<code>dist/</code>	build 後產生的 wheel 和 sdist。通常是執行 build 或下載 GitHub artifact 後才會出現。	Demo 重跑時會用到。若資料夾不存在，先執行 <code>python -m build</code> 或下載 GitHub artifact。
<code>.demo/</code>	本地 rehearsal 產物，例如 local manifest、tampered artifact。	只用於本地練習，不要把它講成正式 GitHub attestation。

5.2 重要檔案地圖

檔案	用途	簡報重點
README.md	專案總說明和 demo 指令。	組員第一次看 repo 先看這份。
pyproject.toml	Python package 設定，定義如何 build wheel。	證明 artifact 是標準 Python package build 產物。
.github/workflows/build-and-attest.yml	正式 GitHub CI/CD 和 attestation 流程。	簡報技術核心。要講 permissions、build、attest、verify。
scripts/verify_artifact.py	本地或 GitHub 模式驗證 artifact。GitHub 模式會呼叫 gh attestation verify。	解釋 verify pass 的 terminal output 從哪裡來。
scripts/tamper_demo.py	複製 artifact，翻轉其中 1 byte，再跑 verify。	解釋 tamper fail 的實作畫面。
demo.ps1	本地 rehearsal 一鍵流程。	如果組員要自己錄 demo，可先跑本地版熟悉流程。
team_handoff/github_evidence.md	GitHub run、commit、artifact、digest 證據摘要。	簡報附錄或報告口頭佐證。
team_handoff/demo_screenshot_checklist.md	截圖 checklist。	做簡報時確認圖沒有漏。
team_handoff/message_to_group.md	可以傳給組員的訊息草稿。	你可以直接複製給組員。

6. Demo 要怎麼重跑

6.1 正式 GitHub attestation demo

這是正式版本。它依賴 GitHub Actions 已經在公開 repo 產生 attestation，然後用 GitHub CLI 驗證下載回來的 artifact。

```
gh auth login
gh run download 26841214975 --name python-distributions --dir dist --repo blizzard-
new/software_supply_chain_provenance_demo
python scripts/verify_artifact.py dist/hello_provenance_demo-0.1.0-py3-none-any.whl --mode github --
repo blizzard-new/software_supply_chain_provenance_demo
python scripts/tamper_demo.py dist/hello_provenance_demo-0.1.0-py3-none-any.whl --mode github --repo
blizzard-new/software_supply_chain_provenance_demo
```

這段 demo 要講的不是「我會用 CLI」，而是「同一個 artifact 可以驗證通過；只要 artifact 內容被改，digest 對不上，GitHub attestation verify 就會失敗」。

6.2 本地 rehearsal demo

如果組員沒有 GitHub CLI 或沒有登入，可以先跑本地 rehearsal。注意：這不是正式 attestation，只是用 local manifest 練習 digest 驗證和 tamper fail。

```
.\demo.ps1
```

或手動執行：

```
python -m pip install build
python -m pip install -e .
python -m unittest discover -s tests
python -m build
python scripts/create_local_manifest.py
python scripts/verify_artifact.py --mode local
python scripts/tamper_demo.py --mode local
```

6.3 Demo 口語講稿

1. 「我先展示 GitHub Actions 成功跑完，包含 test、build、attestation、verify。」
2. 「這個 wheel 是我們的 artifact，它有一個 SHA-256 digest。」
3. 「GitHub attestation 記錄的 subject digest 要和我手上的 artifact digest 一致。」
4. 「現在驗證原始 wheel，結果是 pass。」
5. 「接著我把 wheel 中間某個 byte 翻轉，檔名看起來還是類似，但 SHA-256 已經完全不同。」
6. 「再驗證一次，結果 fail，因為 GitHub 找不到符合這個 digest 的有效 attestation。」


```
● ● ● GitHub attestation verification - PASS

$ python scripts/verify_artifact.py dist/*.whl --mode github --repo blizzard-new/software_supply_chain_provenance
[github] artifact = dist/hello_provenance_demo-0.1.0-py3-none-any.whl
[github] sha256   = 7e1631c43d5ba0743c62a79df5b469df55a73f9c08a972ea1358ba55b3b4a7d5
[github] running = gh attestation verify ... --format json
[github] verification PASSED
[github] subject  = hello_provenance_demo-0.1.0-py3-none-any.whl
[github] digest   = 7e1631c43d5ba0743c62a79df5b469df55a73f9c08a972ea1358ba55b3b4a7d5
[github] builder  = .github/workflows/build-and-attest.yml@refs/heads/main
[github] workflow = build-and-attest
[github] commit   = c7ee14ad9699ec7a942ccc481d6be5aa87a622d9
[github] tlog      = https://rekor.sigstore.dev

Source: GitHub Actions run: 26841214975
```

第 8 頁建議放這張：原始 artifact 驗證通過。

```
● ● ● Tampered artifact verification - FAIL as expected

$ python scripts/tamper_demo.py dist/*.whl --mode github --repo blizzard-new/software_supply_chain_provenance_de
[tamper] original = dist/hello_provenance_demo-0.1.0-py3-none-any.whl
[tamper] tampered = .demo/tampered/hello_provenance_demo-0.1.0-py3-none-any.tampered.whl
[tamper] offset   = 2266
[tamper] original sha256 = 7e1631c43d5ba0743c62a79df5b469df55a73f9c08a972ea1358ba55b3b4a7d5
[tamper] tampered sha256 = 34b3d6cc07aaa308c8ed0723e29dfe107c811f16f49472393162d1df53971242
[tamper] Step 1: verify original artifact. Expected: PASS
[github] verification PASSED
[tamper] Step 2: verify tampered artifact. Expected: FAIL
Error: HTTP 404: Not Found (.../attestations/sha256:34b3d6cc07aa...)
[github] verification FAILED
[tamper] expected result: tampered artifact failed verification

Source: GitHub Actions run: 26841214975
```

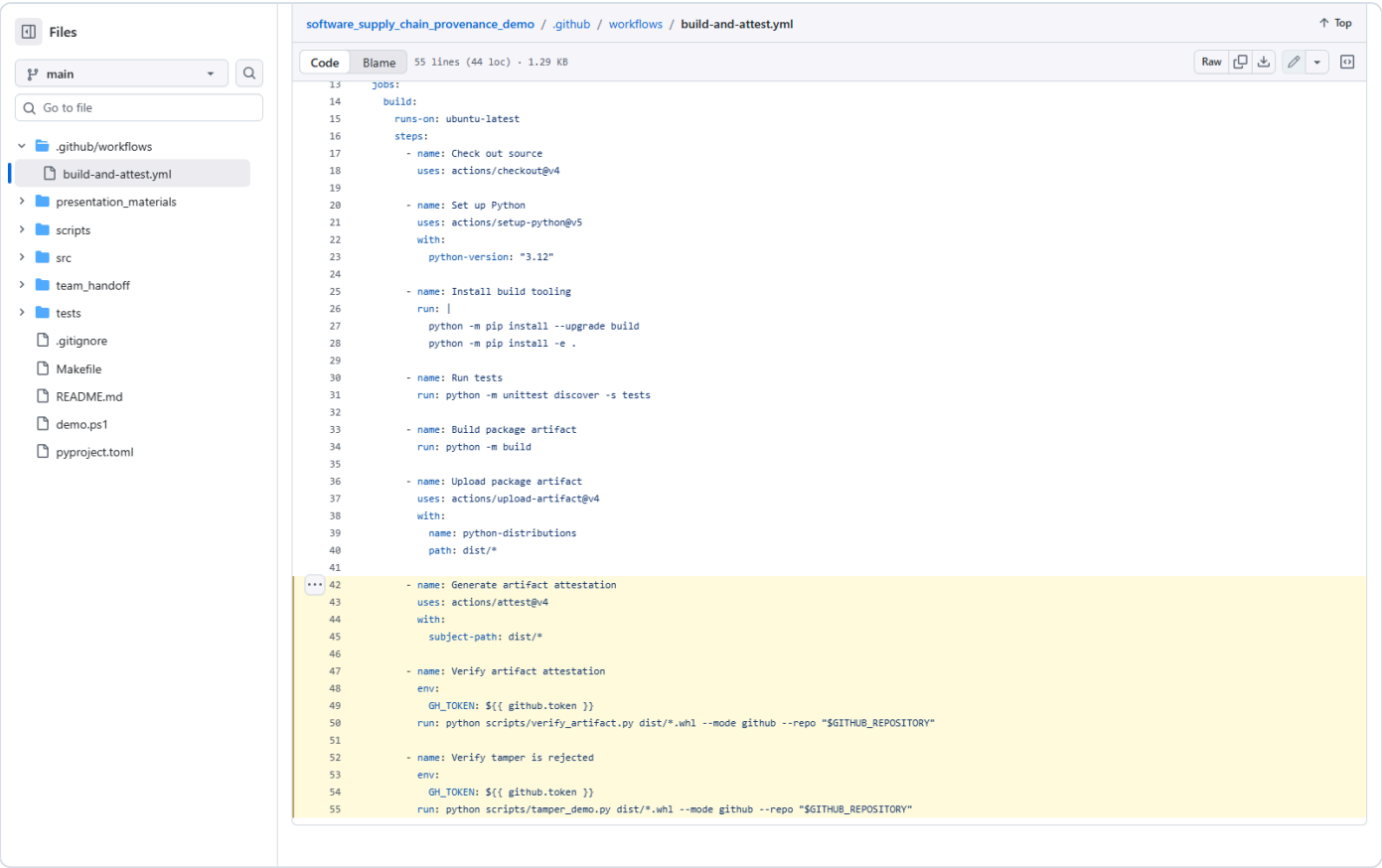
第 9 頁建議放這張：tampered artifact 驗證失敗。

7. 截圖資料夾怎麼用

team_handoff/screenshots/ 已經整理好 12 張圖。組員不需要重新截圖，除非要換風格或補錄 live demo。

截圖	內容	建議 放在 哪一 頁	講法
01_repo_home.png	公開 repo 首頁。	Slide 1 或 最後 附錄。	證明專案已經 push 到公開 GitHub。
02_actions_success.png	GitHub Actions run success。	Slide 5。	展示完整 CI 流程都成功。
03_attestation_created.png	Generate artifact attestation step。	Slide 7。	指出 attestation 是 workflow 產生，不是手寫文字。
04_verify_pass.png	Verify artifact attestation step。	Slide 8 或 備用。	GitHub Actions 裡也有 verify pass。
05_tamper_fail.png	Verify tamper is rejected step。	Slide 9 或 備用。	GitHub Actions 裡確認 tamper 會被拒絕。
06_workflow_code.png	workflow 前半段程式碼。	Slide 6。	講 permissions、test、build、upload artifact。
07_verify_artifact_code.png	verify_artifact.py。	Slide 8 或 附錄。	講驗證腳本如何呼叫 GitHub CLI 並印出 digest、builder、workflow、commit。
08_tamper_demo_code.png	tamper_demo.py。	Slide 9 或 附錄。	講翻轉 1 byte 的 demo 是怎麼做出來的。
09_cli_code.png	CLI source code。	Slide 5 或 附錄。	證明 artifact 有實際 source code，不是空檔案。
10_workflow_attestation_verify_code.png	workflow 後半段。	Slide 7。	講 actions/attest@v4、verify、tamper reject 三個關鍵步驟。

截圖	內容	建議 放在 哪一 頁	講法
11_verify_pass_terminal.png	本機 terminal 顯示 verify pass。	Slide 8。	主 demo 證據。
12_tamper_fail_terminal.png	本機 terminal 顯示 tamper fail。	Slide 9。	主 demo 證據。



這張適合放在第 7 頁，讓聽眾看到 workflow 裡真的有 attestation、verify、tamper reject。

8. 簡報整體架構建議

建議做 12 至 15 頁，報告時間如果是 12 至 15 分鐘，每頁平均 45 至 75 秒。重點是先講問題，再講技術，再講實作證據，最後講限制與延伸。

段落	頁數	目的	不要做的事
開場與問題	1 至 2	讓聽眾知道為什麼 artifact provenance 重要。	不要一開始就丟 workflow code。
概念與趨勢	3 至 4	定義 artifact、provenance、attestation，連到 GitHub / SLSA / PyPI / Sigstore。	不要把所有官方文件逐字塞進投影片。
系統設計	5 至 7	展示本專案架構和 GitHub Actions workflow。	不要逐行念程式碼，只框出關鍵步驟。
Demo 證據	8 至 9	用 verify pass 和 tamper fail 證明有做出可驗證成果。	不要只放文字，這兩頁一定要有 terminal 截圖。
分析與限制	10 至 11	說明 attestation 解決什麼，沒有解決什麼。	不要宣稱 attestation 可以解決所有 supply chain risk。
結論與 Q&A	12 至 15	收斂主軸，附 repo link、分工、Q&A。	不要在結尾加入新技術名詞。

9. 每一頁簡報應該放什麼

Slide 1 - 題目與一句話結論

建議圖片 01_repo_home.png

- **標題：**軟體供應鏈簽章與可驗證 Provenance。
- **畫面：**左側放題目和組員名單，右側放 GitHub repo 首頁截圖。
- **文字：**「我們實作一個 GitHub Actions provenance demo，證明 build artifact 可以被驗證來源，竄改後會驗證失敗。」
- **講稿：**「這次重點不是做一個大系統，而是做一個能被驗證的供應鏈安全流程。」

Slide 2 - 問題背景：為什麼只看 source code 不夠

建議圖 用流程圖：source code -> CI/CD -> artifact -> user。

- **畫面：**用 4 個方塊表示 source、workflow、artifact、下載者。
- **文字：**「攻擊點可能在 build pipeline、artifact 儲存、release 下載，不一定在 source code。」
- **講稿：**「就算 GitHub repo 看起來正常，使用者手上的檔案也可能不是那個 repo 建出來的，所以需要 provenance。」

Slide 3 - 核心概念定義

建議圖 四格表：Artifact、Digest、Provenance、Attestation。

- **畫面：**不要放太多字，每個名詞 1 句解釋。
- **文字：**Artifact 是產物，digest 是內容指紋，provenance 是產生過程，attestation 是可驗證聲明。
- **講稿：**「Provenance 的目的，是把 artifact 連回 build source 和 build process。」

Slide 4 - 現代標準和工具趨勢

建議素材 GitHub / SLSA / PyPI / Sigstore 四個 logo 或文字卡。

- **畫面：**四個卡片，各放一行重點。
- **文字：**GitHub artifact attestations、SLSA build provenance、PEP 740 digital attestations、Sigstore identity-based signing。
- **講稿：**「這個題目不是單一 GitHub 功能，而是整個開源套件生態正在往可驗證 provenance 發展。」

Slide 5 - 本專案實作架構

建議圖片 02_actions_success.png 或自行畫流程圖。

- 畫面：Python CLI package -> test -> build wheel -> upload artifact -> attest -> verify -> tamper fail。
- 文字：「用 GitHub Actions 自動產生和驗證 artifact attestation。」
- 講稿：「我們選 Python wheel 是因為檔案小、build 流程清楚，很適合展示 provenance。」

Slide 6 - GitHub Actions workflow 前半段

建議圖片 06_workflow_code.png

- 畫面：框出 permissions、Run tests、Build package artifact、Upload package artifact。
- 文字：id-token: write 讓 workflow 可以取得 OIDC token；attestations: write 讓 workflow 可以寫入 attestation。
- 講稿：「這些 permissions 是供應鏈安全裡很重要的最小權限設計。」

Slide 7 - Attestation 和驗證 workflow

建議圖片 10_workflow_attestation_verify_code.png

- 畫面：框出 Generate artifact attestation、Verify artifact attestation、Verify tamper is rejected。
- 文字：attest 產生 provenance，verify 驗證原始 artifact，tamper test 驗證竄改後會失敗。
- 講稿：「這真是實作核心，代表 CI 不是只 build，而是把 provenance 也納入驗證流程。」

Slide 8 - Demo 1：原始 artifact 驗證通過

建議圖片 11_verify_pass_terminal.png

- 畫面：放 terminal 截圖，圈出 verification PASSED、subject、digest、builder、workflow、commit。
- 文字：「原始 wheel 的 digest 和 GitHub attestation subject digest 一致。」
- 講稿：「驗證通過不是因為檔名相同，而是 digest 對上 attestation 裡的 subject。」

Slide 9 - Demo 2：竄改 artifact 驗證失敗

建議圖片 12_tamper_fail_terminal.png

- 畫面：放 terminal 截圖，圈出 original sha256、tampered sha256、verification FAILED。
- 文字：「翻轉 1 byte 就會讓 SHA-256 改變，因此找不到符合的有效 attestation。」
- 講稿：「這就是 artifact integrity 和 provenance 綁定後的效果。」

Slide 10 - 實作程式碼畫面

建議圖片 07_verify_artifact_code.png 、 08_tamper_demo_code.png

- 畫面：左右各放一張 code screenshot，不要放太滿。
- 文字：verify_artifact.py 負責驗證；tamper_demo.py 負責製造竄改檔案並確認 fail。
- 講稿：「我們不是只截 GitHub 圖，也有寫驗證和攻擊模擬腳本。」

Slide 11 - 安全性分析：解決了什麼

建議圖 表格或 check list。

- 可以解決：artifact 是否被改、是否來自指定 repo、是否由指定 workflow build、是否能追到 commit。
- 簡報文字：「Provenance 讓驗證者不只看檔案內容，也能看 build context。」
- 講稿：「這讓 release 驗證從『相信發布者』變成『驗證 artifact 與 build 證據』。」

Slide 12 - 限制：沒有解決什麼

建議圖 限制表格。

- 限制 1：如果 source code 本身有惡意邏輯，attestation 仍可能正常通過。
- 限制 2：如果 workflow 被合法 commit 改壞，attestation 只證明它來自那個 workflow，不代表 workflow 本身安全。
- 限制 3：驗證者需要明確 policy，例如接受哪個 repo、哪個 workflow、哪個 organization。
- 講稿：「attestation 不是萬靈丹，它是供應鏈安全裡的 provenance layer。」

Slide 13 - 組員分工與資料夾使用

建議圖 放資料夾表或 repo tree。

- 畫面：列出 presentation_materials/、team_handoff/、team_handoff/screenshots/。
- 文字：「簡報文字看本 PDF，圖片用 screenshots，實作細節看 scripts 和 workflow。」
- 講稿：「這頁給組員和老師知道資料已經整理好，不是只有口頭 demo。」

Slide 14 - 結論

建議圖 用三句收斂。

- 結論 1：現代供應鏈安全要驗證 artifact 的來源與 build process。
- 結論 2：GitHub artifact attestations 可以把 artifact digest 和 SLSA provenance 綁在一起。
- 結論 3：本實作已證明原始 artifact 可驗證通過，竄改 artifact 會失敗。
- 講稿：「我們完成的是一個從 source 到 artifact 到 verification 的端到端 demo。」

Slide 15 - Q&A 和參考資料

建議內容 repo link、官方來源、可能問題。

- 畫面：放 repo link 和 4 至 5 個官方參考來源。
- 可預期問題：「這跟 hash 有什麼不同？」「如果 source code 本來就有毒怎麼辦？」「為什麼要用 GitHub Actions？」
- 講稿：「hash 是內容指紋，provenance 是來源與 build context；兩者一起才比較完整。」

10. 可直接放進簡報的講稿摘要

10.1 開場 30 秒

「我們這組選的題目是軟體供應鏈簽章與可驗證 provenance。核心問題是：當使用者下載一個套件或 binary 時，怎麼知道它真的來自指定 source code 和指定 build workflow？我們用 GitHub Actions 實作了一個 Python artifact demo，產生 GitHub artifact attestation，並用 GitHub CLI 驗證原始 artifact 通過、竄改 artifact 失敗。」

10.2 Demo 前 30 秒

「接下來我們不是只看 source code，而是看 build 出來的 wheel。這個 wheel 有 SHA-256 digest，GitHub attestation 裡也有 subject digest。如果兩者一致，就能驗證它對應到那次 GitHub Actions build。若我把 wheel 改掉 1 byte，digest 會改變，驗證就會失敗。」

10.3 結尾 30 秒

「這個 demo 的價值在於把 release artifact 從『看起來像官方檔案』提升成『可以用 provenance 驗證來源』。它不能取代 code review、dependency scanning 或 SBOM，但可以補上 artifact 來源驗證這一層，讓供應鏈攻擊更難偽裝成正常 release。」

11. 老師可能問的問題與回答

問題	建議回答
你們的 attestation 跟單純 hash 有什麼不同？	hash 只證明檔案內容是否改變；attestation 會把 artifact digest 和來源資訊綁在一起，例如 repo、commit、workflow、builder。兩者不是替代關係，而是 attestation 會使用 digest 作為 subject。
如果 source code 一開始就是惡意的，attestation 會抓到嗎？	不會。attestation 證明 artifact 來自指定 source 和 build process，不保證 source code 本身無漏洞或無惡意。因此仍需要 code review、dependency scanning、測試、SBOM 等配套。
為什麼使用 GitHub Actions，而不是自己本機簽章？	本機簽章會牽涉長期金鑰管理，也很難證明 build context。GitHub Actions 可以用 OIDC 和 GitHub artifact attestations，把 workflow identity 和 build provenance 結合。
本地 manifest 算不算正式 provenance？	不算。本地 manifest 只是課堂 rehearsal，方便示範 digest 驗證和 tamper fail。正式 demo 是公開 GitHub repo 的 workflow 產生 artifact attestation，再用 GitHub CLI 驗證。
你們怎麼證明 tamper 真的被擋？	tamper_demo.py 會複製 artifact 並翻轉其中 1 byte，接著重新跑 verify。terminal 截圖顯示 original sha256 和 tampered sha256 不同，tampered artifact 驗證失敗。
這套流程可以套到 container image 嗎？	可以，GitHub artifact attestations 和 actions/attest 也支援 container image，但 workflow 會改成使用 image name 和 digest。這次選 Python wheel 是為了讓期末 demo 更短、更容易驗證。

12. 參考資料

- GitHub Docs - Using artifact attestations to establish provenance for builds: <https://docs.github.com/en/actions/how-tos/secure-your-work/use-artifact-attestations/use-artifact-attestations>
- GitHub Action - actions/attest: <https://github.com/actions/attest>
- SLSA v1.2 - Provenance: <https://slsa.dev/spec/v1.2/provenance>
- PEP 740 - Index support for digital attestations: <https://peps.python.org/pep-0740/>
- Sigstore Docs - Overview: <https://docs.sigstore.dev/>

交接檢查清單：做 PPT 前請確認至少使用 02_actions_success.png 、
10_workflow_attestation_verify_code.png 、 11_verify_pass_terminal.png 、
12_tamper_fail_terminal.png 。這四張圖足以支撐「有 build、有 attestation、有 verify、有 tamper fail」。